```
spotless = false
until spotless
  clean 'Everything'
  spotless = true
```

This until loop exits the first time around. Without assigning true to the spotless variable, the loop won't exit.

Some things never end, though, and that's what the `loop` keyword is for:

```
loop clean 'Everything'
```

If you run this on the REPL, it will eventually exit with FATAL ERROR: JS Allocation failed - process out of memory.

Loops are expressions, so they have a value, and if you have a loop that terminates, you can get the value:

```
x = 0
evenNumbers = while x < 6
  x = x + 1
  x * 2

evenNumbers
# [2, 4, 6, 8, 10, 12]
```

Most likely you won't use these looping constructs in CoffeeScript very often. They are there if you need them, though.

Finally, if you really need to, you can get out of a `while`, `until`, or `for` by using the `break` keyword. If you aren't familiar with the `break` keyword, then happily move on.

Exception blocks

These are provided by the `try`, `catch`, and `finally` keywords. Use them to deal with *exceptional* circumstances inside a block of code. You should use the `finally` block to clean up after yourself. Exceptions are created with the `throw` statement. A `try` can also have a `catch`, a `finally`, or both:

```
flyAway = (animal) ->
  if animal is 'pig'
    throw 'Pigs cannot fly'
  else
```